Hotel Revenue Manager

UML Design: use case, sequence and class diagrams

Repository: github.com/stanleyreubens/565_project | Author: Stanley Ogbumuo | Iteration 1 | Full resolution PNG and SVG copies of every diagram are in **uml-diagrams/**

Hotel Revenue Manager will recommend a nightly rate for every room type on every date, priced against live competitor rates, local event demand and the hotel's own cost floor. This document models that system three ways: what it will do and for whom, how its parts will collaborate to answer the central question, and how its data and behaviour will be structured.

How to read these diagrams

The three views answer different questions and are meant to be read together. The use case diagram fixes the scope and the boundary, naming who sits outside the system and what they will be able to ask of it. The sequence diagram takes the single most important of those cases and follows one request through every object that touches it, in order. The class diagram steps back from any one request and shows the structure that makes all of them possible.

Every diagram in this document is vector art. Zooming in will not blur the text, and the same drawings are supplied as 200 dpi PNG files alongside their PlantUML source.

1. Use case diagram

The boundary and the actors

The rectangle labelled Hotel Revenue Manager is the system boundary. Everything inside it is behaviour the project will build. Everything outside it is an actor, meaning something that exchanges information with the system without being part of it.

Three of the five actors are human. The **Revenue Manager** is the primary actor and starts most of the work. The **Organisation Administrator** is drawn as a generalisation of the Revenue Manager, using the hollow triangle: an administrator can do everything a revenue manager can, and additionally sees every property the organisation owns. The **Platform Administrator** operates the service and appears only against configuration.

The remaining two are external systems. The **Rate Provider** supplies live competitor rates and the property catalogue used when importing. The **Events Service** supplies what is happening in the city. Both are drawn as actors because they sit outside the boundary and the project does not build them.

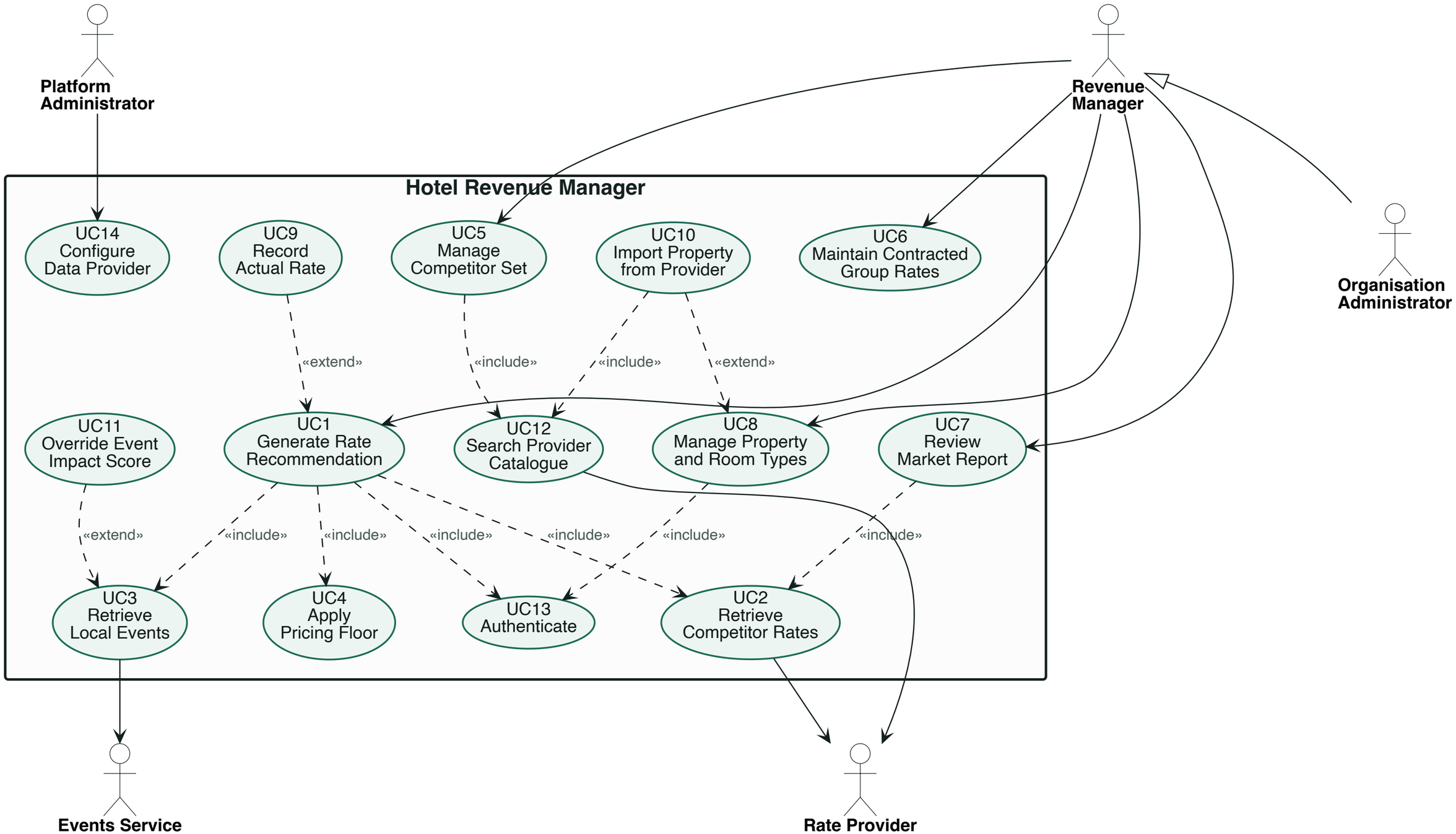
Include and extend

The two relationships mean different things and are used deliberately here. An **«include»** is mandatory: the base use case is incomplete without it. UC1 cannot produce a recommendation without retrieving competitor rates, reading the events calendar and applying the floor, so all three are included. Factoring them out also stops the same behaviour being described twice, which is why UC2 is included by both UC1 and UC7, and UC12 by both UC5 and UC10.

An **«extend»** is optional and points the other way, from the extending case to the base. UC9 extends UC1 because recording what was actually charged is something the manager may do afterwards, and a recommendation is complete whether or not they do. UC10 extends UC8 for the same reason: a property can be created by hand, and importing it is an alternative path. UC11 extends UC3, since an event carries a default score and overriding it is a choice.

> **A note on modelling authentication.** UC13 is included by UC1 and UC8 rather than by all twelve remaining cases. Drawing it from every one would be strictly accurate but would bury the diagram in lines that tell the reader nothing new. A valid session is a precondition of every use case except UC13 itself, and that is stated here in text instead.

Figure 1. Use case diagram. The rectangle is the system boundary; the five actors sit outside it. Dashed arrows carry «include» and «extend» stereotypes, and the hollow triangle between the two administrator actors is a generalisation.



Use case descriptions

UC1 Generate Rate Recommendation

Primary actor: Revenue Manager
Includes UC2, UC3, UC4 and UC13. Extended by UC9.

The manager picks a hotel, a room type and a stay window. The system will gather what competitors are charging, find the events falling on those dates, apply the hotel's floor, and return a recommended nightly rate with the alternative strategies and the figures behind them. This is the use case the product exists for, and the one the sequence diagram expands.

UC2 Retrieve Competitor Rates

Primary actor: Rate Provider (supporting)
Included by UC1 and UC7. Communicates with the Rate Provider.

For a given stay window the system will request live room rates for every competitor attached to the hotel, normalise the replies, and discard any rate that is missing or non numeric. It never runs on its own, which is why no actor starts it directly.

UC3 Retrieve Local Events

Primary actor: Events Service (supporting)
Included by UC1. Extended by UC11. Communicates with the Events Service.

The system will fetch concerts, conferences and fixtures taking place in the hotel's city across the dates being priced, and give each one an impact score between 1 and 10 that drives the demand uplift.

UC4 Apply Pricing Floor

Primary actor: none (supporting)
Included by UC1.

Before a recommendation is returned it will be tested against a floor. The floor is the cost per night plus the minimum margin, unless a contracted group rate covers the date and sits higher, in which case that rate becomes the floor.

UC5 Manage Competitor Set

Primary actor: Revenue Manager
Includes UC12.

The manager searches for rival properties and attaches them to a hotel, keeping the provider identifier, star rating and distance. Competitors can be removed when they stop being relevant. The set is what every later comparison is measured against.

UC6 Maintain Contracted Group Rates

Primary actor: Revenue Manager
No include or extend relationships.

Rates already negotiated with a group are recorded against a date range and a room type, with the group name, the segment and any notes. The system will refuse a range whose end date falls before its start.

UC7 Review Market Report

Primary actor: Revenue Manager
Includes UC2.

The manager sees the hotel's rate plotted against the competitor average across a chosen date range, which shows whether the property has drifted expensive or cheap over a season rather than on one night.

UC8 Manage Property and Room Types

Primary actor: Revenue Manager
Includes UC13. Extended by UC10.

Hotels are created and edited here, along with the room types each one sells and their base prices. A hotel that is no longer managed is marked inactive rather than deleted, so its pricing history survives.

UC9 Record Actual Rate

Primary actor: Revenue Manager
Extends UC1 at the point where a recommendation has been presented.

After acting on a recommendation the manager may come back and record the rate actually loaded into the booking system, together with the occupancy achieved. This is optional behaviour layered on top of UC1, which is why it extends rather than includes.

UC10 Import Property from Provider

Primary actor: Revenue Manager
Extends UC8. Includes UC12.

Rather than typing a property in by hand, the manager may search a travel agency listing and import the match, which creates the hotel and, where the provider returns them, its room types. UC8 is complete without it, so this is an extension.

UC11 Override Event Impact Score

Primary actor: Revenue Manager
Extends UC3.

Where the manager knows more about the local market than the listing does, the impact score of an event can be raised or lowered, which changes every recommendation for the affected dates.

UC12 Search Provider Catalogue

Primary actor: Rate Provider (supporting)
Included by UC5 and UC10. Communicates with the Rate Provider.

A shared step that queries a provider for properties matching a name and city. Both competitor selection and property import need it, which is why it is factored out and included by each.

UC13 Authenticate

Primary actor: Revenue Manager
Included by UC1 and UC8. See the note on modelling below.

Every request carries a signed token that the system will verify before any hotel data is read, loading the user together with their organisation so that access can be scoped. A missing or expired token returns the manager to the sign in page.

UC14 Configure Data Provider

Primary actor: Platform Administrator
No include or extend relationships.

The platform administrator selects which competitor data source is active. The choice is made through configuration rather than code, so the source can be changed without a release.

2. Sequence diagram

Why this use case

UC1 Generate Rate Recommendation is modelled because it generalises the product. It is the only case that touches every part of the system at once: the interface, authentication, both external providers, the database and the pricing calculation itself. Anyone who understands this one interaction understands the shape of the software. It runs across two sheets because a single sheet at this level of detail would shrink the type past the point of being readable.

Participants

The actor is the Revenue Manager. Inside the boundary box the objects are stereotyped by responsibility: **PricingPage** is the boundary object the manager interacts with, **PricingRoute** is the control object that receives the HTTP request, and **AuthService**, **CompetitorProvider**, **EventsService** and **PricingService** are the controls it delegates to. **PostgreSQL via Prisma** is drawn with the database stereotype. The **Rate Provider API** and **Events API** sit outside the box, matching the two external actors in Figure 1.

Message types

Four kinds appear. A **synchronous** call is a solid line with a filled head, and the caller waits: most of the diagram is these. A **reply** is a dashed line with an open head, carrying the value back. An **asynchronous** message is a solid line with a thin open head, used for the two calls that leave the system over HTTPS, where the provider is free to take as long as it takes. A **self message** loops back to its own lifeline, used where an object does internal work rather than calling out, such as normalising a payload or computing the competitor average. The tall narrow boxes on each lifeline are activation bars, showing when an object holds control.

Interaction fragments

Four fragment types are used, all of them load bearing rather than decorative.

alt in part 1 splits on whether the session is valid. The first operand returns 401 and sends the manager back to sign in; everything else in the use case sits in the second operand, which is the honest way to draw it, since none of that work happens for an unauthenticated request.

par shows the two market signals being gathered at the same time. Competitor rates and events are independent, so waiting for one before starting the other would add delay for no reason.

loop sits inside the par and repeats the rate request once per competitor in the set.

opt is used twice for single conditional paths: once in part 2 where the provider is unreachable and the last known figures are read from the database instead, and once inside PricingService where a contracted group rate sits above the cost floor and becomes the floor. A conditional with one operand is an opt rather than an alt, which is why the fallback is drawn that way.

Figure 2. Sequence diagram for UC1 Generate Rate Recommendation, part 1 of 2: the request arrives, the session is verified, and the two market signals are gathered in parallel.

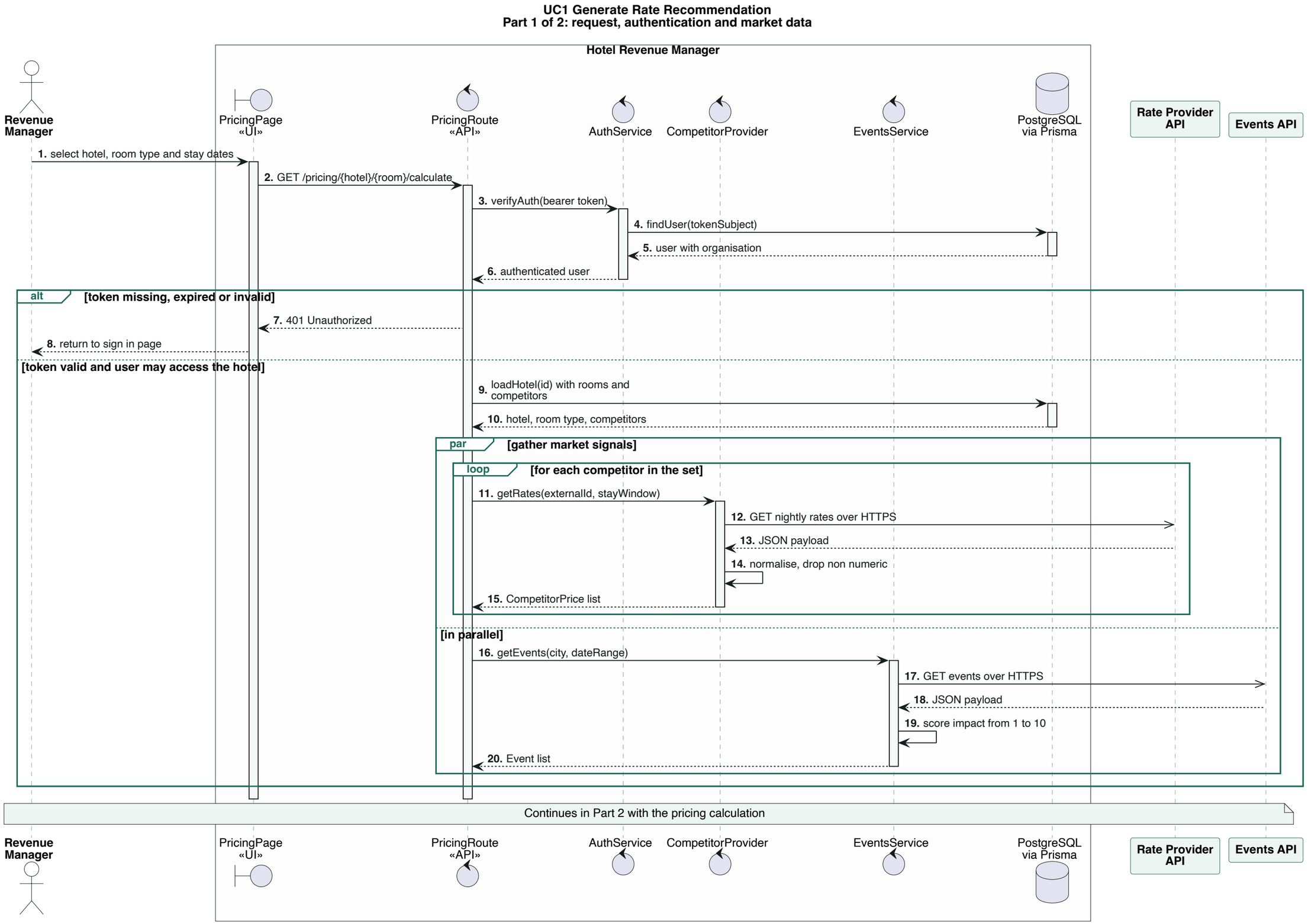
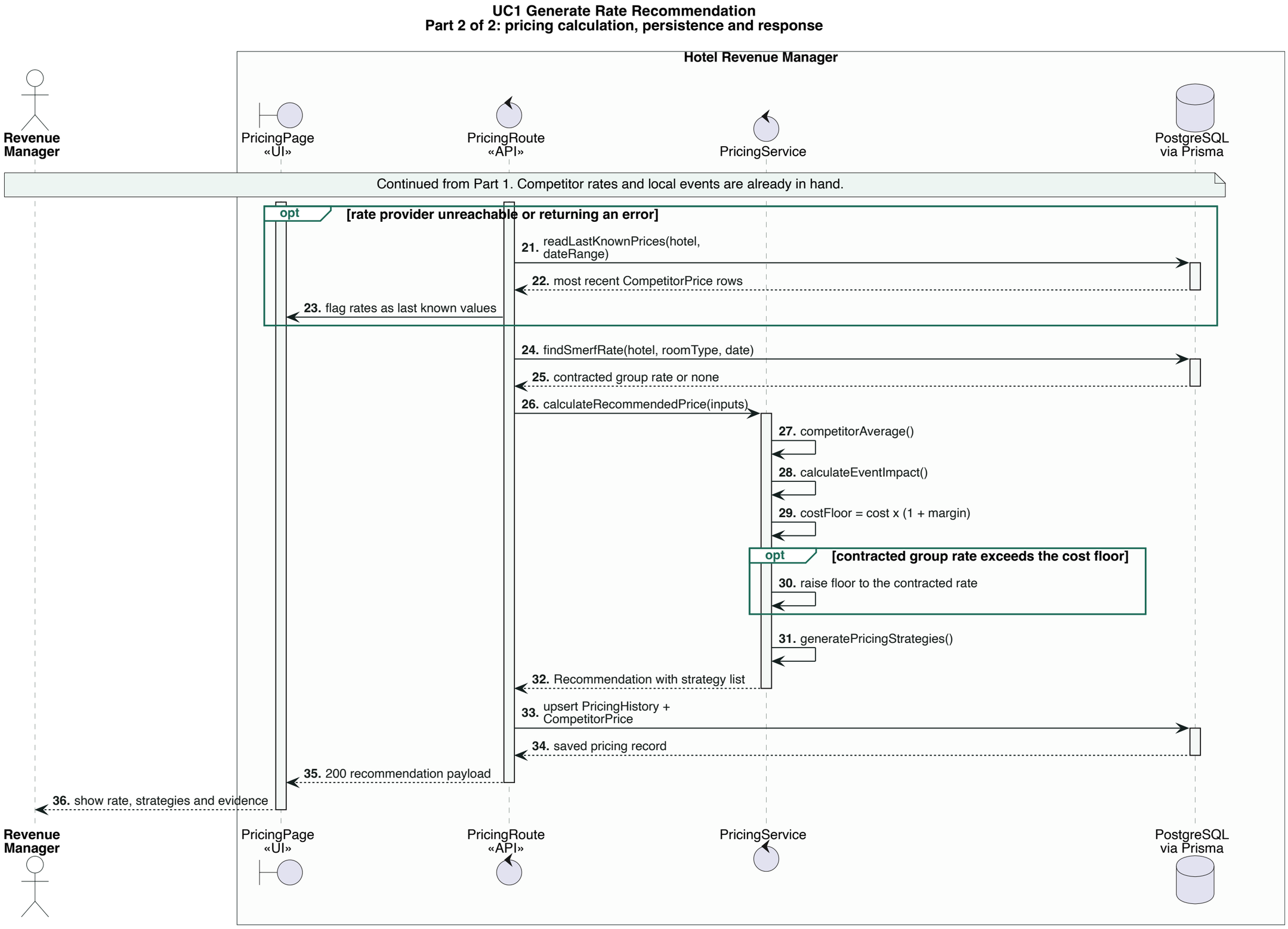


Figure 3. Sequence diagram for UC1, part 2 of 2: the floor is resolved, the rate is calculated, the result is persisted and the answer returns to the manager. Message numbering continues from part 1.



3. Class diagram

Structure

The classes fall into three groups. **Domain** holds the things the business talks about, and these map one to one onto the database tables. **Services** holds the classes that do work rather than hold state. **Value objects** holds the results the pricing calculation produces, which are returned to the caller and never stored in their own right.

Attributes carry visibility and type. Fields are private and reached through behaviour, which is why `passwordHash` is never exposed and `verifyPassword` is offered instead. Methods are listed where a class has real behaviour, such as `Hotel.deactivate`, `SmerfRate.coversDate` and `PricingHistory.recordActual`.

Choosing between relationship types

Composition, the filled diamond, is used where the part cannot exist without the whole. A RoomType is meaningless without its Hotel, and a CompetitorPrice is meaningless detached from the PricingHistory row it justifies. Deleting the whole deletes the parts.

Aggregation, the hollow diamond, is used between Organization and both User and Hotel. An organisation groups them, but a user record would survive being moved to another tenant, so the tie is weaker.

Association, a plain line, covers the rest, including the many to many between PricingHistory and Event.

Realisation, the dashed line with a hollow triangle, connects BookingProvider and SerpApiProvider to the CompetitorProvider interface. This is what lets the data source be swapped by configuration: PricingService depends on the interface and never on a particular provider.

Dependency, a dashed arrow, marks a class that uses another without holding a reference to it, such as PricingService creating a Recommendation. Realisation and dependency carry no multiplicities, because neither describes a set of linked instances.

Multiplicities

FROM	CARD.	CARD.	TO	KIND	WHY IT IS MODELLED THIS WAY
`Organization`	`1`	`0..*`	`User`	aggregation	An organisation groups users, but a user record could be moved and would still be meaningful.
`Organization`	`1`	`0..*`	`Hotel`	aggregation	Hotels belong to a tenant without being destroyed by it.
`User`	`1`	`0..*`	`Hotel`	association	The creator of a hotel is recorded so an independent account can be scoped to its own properties.
`Hotel`	`1`	`1..*`	`RoomType`	composition	A room type has no meaning apart from its hotel, and a hotel must sell at least one.
`Hotel`	`1`	`0..*`	`Competitor`	composition	A competitor entry exists only as part of one hotel's set.
`Hotel`	`1`	`0..*`	`SmerfRate`	composition	A contracted rate dies with the property it was contracted for.
`Hotel`	`1`	`0..*`	`PricingHistory`	association	Priced nights are kept for reporting after the hotel is deactivated.
`RoomType`	`1`	`0..*`	`PricingHistory`	association	Each priced night belongs to exactly one room type.
`RoomType`	`1`	`0..*`	`SmerfRate`	association	A contracted rate covers one room type.
`PricingHistory`	`1`	`0..*`	`CompetitorPrice`	composition	The observed rates are the evidence for one recommendation and are meaningless detached from it.
`PricingHistory`	`0..*`	`0..*`	`Event`	association	One conference affects every room type priced for that night, and one priced night may be driven by several events.
`Hotel`	`0..1`	`0..*`	`Reminder`	association	A reminder may concern a property or may be general to the organisation.
`Recommendation`	`1`	`1..*`	`Strategy`	composition	A recommendation always carries at least the Balanced strategy.

The remaining links are each exactly one to one: a User has one Role, an Event has one EventSource, and a Reminder has one ReminderPriority and one ReminderStatus. These are enumerations rather than entities, so the multiplicity is fixed by definition.

Two constraints in the diagram are worth calling out because they encode business rules rather than structure. Hotel to RoomType is `1..*` at the room end, since a property that sells no room types cannot be priced and should not be accepted. PricingHistory to Event is many to many, which is what allows a single conference to raise the recommendation for every room type on that night while a single night can be driven by several events at once.

Figure 4. Class diagram. Domain classes on the left, application services centre, value objects right. Every association carries multiplicities at both ends; filled diamonds are composition and hollow diamonds aggregation.

